Figure 11.2 An authorization server (AS) typically stores client details in a private database, so these details are not accessible to APIs. A service account lives in the shared user repository, allowing APIs to look up identity details such as role or group membership.

One solution to this problem is for the API to make access-control decisions purely based on the scope or other information related to the access token itself. In this case, access tokens act more like capability tokens discussed in chapter 9, where the token grants access to resources on its own and the identity of the client is ignored. Fine-grained scopes can be used to limit the amount of access granted.

Alternatively, the client can avoid the client credentials grant and instead obtain an access token for a *service account*. A service account acts like a regular user account and is created in a central directory and assigned permissions and roles just like any other account. This allows APIs to treat an access token issued for a service account the same as an access token issued for any other user, simplifying access control. It also allows administrators to use the same tools to manage service accounts that they use to manage user accounts. Unlike a user account, the password or other credentials for a service account should be randomly generated and of high entropy, because they don’t need to be remembered by a human.

> **DEFINITION** A ***service account*** is an account that identifies a service rather than a real user. Service accounts can simplify access control and account